

CPF | Core Platform Framework

Business Platform Documentation

CPF 기술사양서

Runtime·Starter·Provider·DB·Integration·Batch 의 기술 사양과 호환성 기준을 정리합니다.

문서 기준

Documentation Harness v2.1.0

기준 Source

master 054d894b47f4be8323439dc6f9e58b7d8b60fe54

기준일

2026-08-26

이 문서는 최신 Source 의 공개 계약과 운영 경계를 기준으로 작성하며, 실행하지 않은 검증을 성공으로 기록하지 않습니다.

목차

역할과 질문을 기준으로 필요한 절을 빠르게 찾습니다.

1. Runtime · Java · Spring · Build 기준

- 1.1 Runtime Baseline
- 1.2 Java
- 1.3 Spring/Spring Boot
- 1.4 Build/Artifact

2. Starter · Profile · Provider · Configuration

- 2.1 Starter
- 2.2 Profile
- 2.3 Provider
- 2.4 Configuration

3. Common · Persistence · Transaction

- 3.1 Framework Common
- 3.2 Persistence
- 3.3 Transaction

4. Integration · Messaging · Notification · File

- 4.1 Integration
- 4.2 Messaging
- 4.3 Notification
- 4.4 File/Archive/Object Storage

5. Security · Observability

- 5.1 Security

- 5.2 Audit/Approval

- 5.3 Observability

6. Platform Operations · OpenAPI

- 6.1 ADM/Platform Operations

- 6.2 OpenAPI

- 6.3 Frontend Client

7. Batch · Gateway

- 7.1 Batch

- 7.2 Gateway

8. Database · Generator · Generated Domain

- 8.1 Database

- 8.2 Generator

- 8.3 Generated Domain

9. Performance · Compatibility · Release

- 9.1 Performance/Capacity

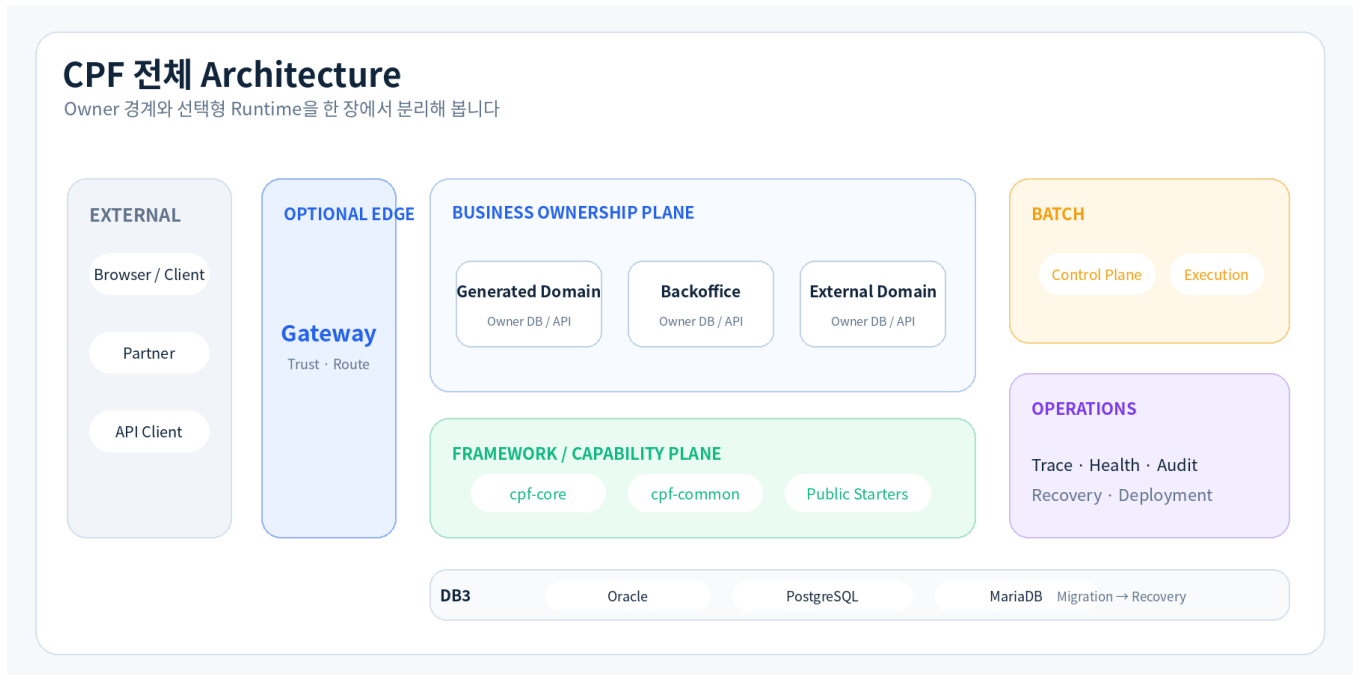
- 9.2 Compatibility

- 9.3 Build/Test

- 9.4 Release Acceptance

1. Runtime·Java·Spring·Build 기준

그림 1-1. CPF Runtime 구성



Business Domain 과 Framework Capability, 운영 Runtime 의 물리적 역할을 구분합니다.

1.1 Runtime Baseline

CPF Runtime baseline 은 Java 25, Gradle 9.1.0, Spring Boot 4.1.0 입니다. root build, Starter, Generated Domain 과 publication 결과가 같은 stack 을 사용해야 합니다.

```
javaVersion=25
```

```
gradleVersion=9.1.0
```

```
springBootVersion=4.1.0
```

1.2 Java

Java 25 toolchain 을 공식 기준으로 사용하며 compile/test/runtime 에서 다른 JDK fallback 으로 성공을 만들지 않습니다.

공식 Java baseline 은 25 이며 root build 와 generated domain compile/test 가 동일 toolchain 에서 통과해야 합니다.

1.3 Spring/Spring Boot

Spring Boot 4.1.0 을 기준으로 AutoConfiguration, Starter 조건과 Provider wiring 을 검증합니다. 선택되지 않은 Capability 가 부수 bean 을 만들지 않아야 합니다.

Spring Boot baseline 은 4.1.0 이며 Starter AutoConfiguration 과 Provider wiring 은 이 baseline 에서 검증합니다.

1.4 Build/Artifact

Build/Compatibility 는 Java·Gradle·Spring baseline 과 Artifact Catalog 를 정본으로 맞춥니다.

Publication/BOM/SBOM/verification 결과가 다른 Source snapshot 을 가리키지 않게 합니다.

- 현재 baseline 은 Java 25, Gradle 9.1.0, Spring Boot 4.1.0 입니다.
 - 공개 Surface 변경은 consumer compile/runtime 과 generated output 까지 회귀 검증합니다.
- Java/Gradle/Spring baseline 과 publication Artifact 가 같은 버전 체계를 사용하는지 확인합니다.

표 1-1. 항목 / 지원/기준 / 제약 / 확인

비교·선택·확인에 필요한 항목만 압축해 정리합니다.

항목	지원/기준	제약	확인
Java	25	프로젝트 baseline 일치	gradle/cpf-stack.properties
Gradle	9.1.0	Wrapper 기준	gradle/cpf-stack.properties
Spring Boot	4.1.0	CPF baseline	gradle/cpf-stack.properties
DB	Oracle·PostgreSQL·MariaDB	DB3 only	cpf-tools/db/vendor

Source [길찾기 gradle/cpf-stack.properties](#)

2. Starter·Profile·Provider·Configuration

2.1 Starter

Starter 는 Application 이 필요한 Capability 를 명시적으로 선택하는 Public Entry 입니다. Provider 는 명시 선택하고 충돌 시 fail-fast 하며 Internal leaf 는 개발자 선택 Surface 로 노출하지 않습니다.

- 한 Deployable 은 기본적으로 exactly-one Top-level Profile 을 선택합니다.
- 선택하지 않은 Optional Capability 는 bean/thread/config/sql/endpoint Side Effect 가 없어야 합니다.

2.2 Profile

Profile 은 Deployable 의 공개 실행 형태와 Capability 묶음을 선택합니다. 하나의 Deployable 에서 상충하는 Top-level Profile 을 동시에 활성화하지 않습니다.

Profile 은 Deployable 의 공개 실행 형태를 선택하며 하나의 실행 단위가 상충하는 Top-level Profile 을 동시에 활성화하지 않게 합니다.

2.3 Provider

Provider 는 같은 Public Capability 의 구현체를 선택하는 확장점입니다. 중복 Provider 가 활성화되면 fail-fast 하고 Internal leaf 를 고객 선택 Surface 로 노출하지 않습니다.

선택되지 않은 Provider 는 bean, thread, config, SQL, endpoint 를 남기지 않습니다.

2.4 Configuration

Config 와 운영 State 는 Source 기본값, environment/profile, 정책/override 의 소유권과 우선순위를 분리합니다. Dynamic 변경은 적용 대상·version·reason·rollback 가능성을 추적합니다.

- secret 값을 일반 property dump 나 ADM 화면에 노출하지 않습니다.
- 동적 변경은 다중 인스턴스 전파와 stale version 충돌을 검증합니다.

필요 Capability 에 맞는 Starter/Provider 를 선택합니다.

표 2-1. 필요 기능 / Starter/Provider / 설정 / 선택 기준

비교·선택·확인에 필요한 항목만 압축해 정리합니다.

필요 기능	Starter/Provider	설정	선택 기준
Web API	cpf-starter-web-api	HTTP/Context	Online API
Secure API	cpf-starter-secure-api	Security	보호 API
Persistence	data-jdbc/mybatis/jpa	Provider 1 개	DB 접근
Messaging	kafka/rabbitmq/jms/ibm-mq	Broker 선택	비동기 연계
Cache	caffeine/redis/valkey	TTL/invalid.	cache 필요 시

기본값이 아닌 실제 선택이 필요한 옵션을 확인합니다.

[Source 길찾기 cpf-tools/generator/contracts/cpf-starter-catalog.json](#)

3. Common·Persistence·Transaction

3.1 Framework Common

Framework Common 은 Logging, Validation, Code/Parameter/Message 와 같은 반복 기능의 Public 계약을 제공하고 Starter 가 Runtime wiring 을 담당합니다.

Framework Common 은 고객 업무 코드가 반복 구현하지 않아야 할 공통 계약을 제공하고 Starter 가 runtime wiring 을 담당합니다.

3.2 Persistence

Persistence 는 JDBC/MyBatis/JPA 중 업무와 팀 표준에 맞는 Provider 를 선택하면서 Repository·Paging·Bulk·Lock 의 공통 계약을 유지합니다.

- Domain 은 다른 Owner 의 Repository/DB 를 직접 조회하지 않습니다.
- Bulk/Paging 은 transaction 범위·lock·index·memory pressure 를 함께 검토합니다.

3.3 Transaction

Transaction 사양은 local DB 원자성, propagation, idempotency, remote Side Effect 와 UNKNOWN/Reconcile 경계를 구분합니다.

Local Transaction 과 Remote Side Effect 의 경계를 분리하고 분산 결과가 확정되지 않으면 UNKNOWN·Reconcile 계약을 사용합니다.

작업에 사용할 Public API 와 실패 주의점을 확인합니다.

[Source 길찾기 cpf-starters/data/persistence/src/main/java/com/cpf/data/persistence/api/transaction/CpfTransactionOperations.java](#)

4. Integration·Messaging·Notification·File

4.1 Integration

Integration 은 HTTP·Fixed-Length·GraphQL·Realtime·Webhook 등 외부 연계 방식의 timeout, credential, retry 와 결과 확정 의미를 정의합니다.

Integration 은 HTTP·Fixed-Length·GraphQL·Realtime·Webhook 등 Provider 별 호출 방식과 timeout·retry·UNKNOWN 의미를 같은 Public 계약 아래 둡니다.

4.2 Messaging

Messaging 은 Broker 종류와 무관하게 schema, producer/consumer context, duplicate, DLQ 와 replay 를 함께 관리합니다. 전송 성공과 업무 처리 성공을 같은 상태로 보지 않습니다.

- Outbox/Inbox 를 사용할 때 DB commit 과 publish/consume 상태를 분리합니다.
- Replay 는 idempotency 와 ordering/partition 영향을 확인한 뒤 수행합니다.

4.3 Notification

Notification 은 발송 요청, Provider 결과, Receipt, Reconcile 상태를 분리합니다. 요청 수락만으로 최종 전달 성공을 확정하지 않습니다.

- Provider 별 message/receipt ID 를 내부 execution 과 연결합니다.
- UNKNOWN/지연 상태는 중복 발송 전에 조회·Reconcile 합니다.

4.4 File/Archive/Object Storage

File 기능은 Attachment, Object Storage, Archive/Checksum/Tabular 처리를 선택형 Capability 로 제공합니다. 파일 원문과 metadata, scan/검증 상태를 분리합니다.

- checksum·content type·size·inspection 결과를 처리 전 검증합니다.
- Object Storage 장애/partial upload 는 reconcile/cleanup 경로를 가집니다.

File/Archive/Object Storage 의 지원 Provider 와 크기·checksum·실패 제약을 확인합니다.

[Source 길찾기](#) `cpf-starters/file/src/main/java/com/cpf/file/objectstorage/api/CpfObjectStorageOperations.java`

5. Security·Observability

5.1 Security

Security 는 Authentication 이후 Authorization/Permission 을 분리해 평가하고 Session/Token lifecycle 을 Runtime 경계에 맞춥니다. 관리 기능과 업무 기능은 서로 다른 권한 Surface 를 가집니다.

- Protected Header 와 사용자 credential 을 동일 identity 로 취급하지 않습니다.
- 401/403 은 원인을 구분하고 실패도 audit/trace 와 연결합니다.

5.2 Audit/Approval

위험 운영 조치는 Permission 만으로 끝내지 않고 reason, 필요 시 approval, 실행 결과와 audit 를 연결합니다. Break-glass 는 일반 권한 우회가 아니라 별도 통제된 비상 절차입니다.

- 승인자와 실행자의 SoD 를 적용할 수 있어야 합니다.
- 실행 실패도 승인/요청/결과 trace 에서 누락하지 않습니다.

5.3 Observability

Observability 는 transactionId·operationId·instanceId 를 중심으로 Log, Trace, Metric, Timeline 을 연결합니다. 운영자는 코드 위치보다 거래·실행 식별자에서 시작해 원인을 좁힐 수 있어야 합니다.

- 민감정보는 structured log 에서도 마스킹합니다.
- 재시도/복구 segment 가 된 거래와 단절되지 않도록 correlation 을 유지합니다.

Log·Trace·Metric correlation 과 masking/운영 조회 지원 범위를 확인합니다.

Source [길찾기](#) [cpf-starters/platform-operations/observability/src/main/java/com/cpf/platform/operations/observability/api/CpfTelemetry.java](#)

6. Platform Operations·OpenAPI

6.1 ADM/Platform Operations

ADM/Platform Operations 는 Runtime 상태·정책·Trace·위험 조치를 조회/제어하는 운영 Surface 이며 Permission·Reason·Approval·Audit 를 연결합니다.

Platform Operations 는 Runtime 상태·정책·Trace·위험 조치를 운영 Surface 로 제공하며 권한·사유·감사 결과를 함께 남깁니다.

6.2 OpenAPI

OpenAPI 는 Controller 계약의 operationId 와 오류/DTO 를 외부 Consumer Surface 로 고정합니다. Frontend 는 Generated Client 를 실제 화면 Consumer 에 연결하고 수기 중복 API wrapper 를 줄입니다.

- 401·403·404·409·429·500·503 상태를 화면/운영 흐름에서 구분합니다.
- Generated client 가 stale 하지 않도록 Source→OpenAPI→client→route coverage 를 검증합니다.

6.3 Frontend Client

Frontend Client 는 최신 OpenAPI 에서 생성된 타입·operation 을 사용하고 실제 화면 Route/Store 가 Generated Client 를 소비해야 합니다. 수기 duplicate wrapper 로 계약을 다시 만들지 않습니다.

- 401·403·404·409·429·500·503 상태를 화면/운영 흐름에서 구분합니다.
- Backend operation 변경 시 OpenAPI regenerate→client compile→화면 route consumer 까지 같은 변경으로 검증합니다.

OpenAPI Generated Client 와 실제 Frontend Consumer 의 호환 조건을 확인합니다.

Source [길찾기](#) [cpf-admin/frontend/src/generated/orval](#)

7. Batch·Gateway

7.1 Batch

Batch 사양은 Control Plane, Scheduler, Worker, Center-Cut, Agent 와 Job/Execution 상태·lease/fencing·복구 계약을 포함합니다. 실행 topology 는 LOCAL·PARALLEL_STEPS·LOCAL_PARTITION 으로 제한합니다.

Batch runtime 은 cpf-batch 가 소유하며 Control Plane, Scheduler, Worker, Center-Cut, Agent 의 역할과 상태를 분리합니다. Batch 전용 Kafka/Broker Remote Transport 는 제품 범위가 아닙니다.

7.2 Gateway

Gateway 는 외부 진입·trust normalization·routing·rate/resilience 를 담당하는 Optional Edge 입니다. 업무 로직과 Business DB 는 Gateway 에 두지 않습니다.

- Gateway 없이 배포하는 topology 도 공식 Domain 계약을 유지합니다.
- Route 변경은 health/draining/canary/rollback 과 함께 운영합니다.

Gateway Optionality, route/resilience 와 비사용 topology 의 지원 조건을 확인합니다.

Source [길찾기](#) `cpf-gateway/src/main/java/com/cpf/gateway/api/CpfGatewayRoute.java`

8. Database·Generator·Generated Domain

8.1 Database

Database 는 Oracle·PostgreSQL·MariaDB 를 공식 Vendor 로 지원하고 Canonical Source→Migration→Upgrade→Rollback/Recovery→Runtime Query parity 를 유지합니다.

공식 DB Vendor 는 Oracle, PostgreSQL, MariaDB 이며 Canonical Source 부터 Migration·Upgrade·Rollback/Recovery·Runtime Query 까지 DB3 parity 를 유지합니다.

8.2 Generator

Generator 는 Canonical Catalog 와 cpf-domain 계약에서 Generated Domain 구조, Starter 와 DB 연결을 생성하고 Generated/User-owned 경계를 보존합니다.

Generator 는 Canonical Catalog 와 cpf-domain 계약을 기준으로 Domain 구조와 Starter/DB 연결을 생성하며 Generated/User-owned 경계를 보존합니다.

8.3 Generated Domain

Generated Domain 은 Project→Runtime Module→Java Base Package→Business Feature→Technical Role 순서로 구조를 분리합니다. Runtime 이름이나 Domain 이름을 package/feature 에 중복 생성하지 않습니다.

- base/는 Generator-owned bootstrap/common contract 에만 사용합니다.
- Feature 에 필요하지 않은 controller/service/repository role 을 기계적으로 만들지 않습니다.

Generated Domain 이 참조하는 DB3 binding 과 Vendor-neutral 설정이 Canonical Catalog 와 일치하는지 확인합니다.

표 8-1. 항목 / Oracle / PostgreSQL / MariaDB

비교·선택·확인에 필요한 항목만 압축해 정리합니다.

항목	Oracle	PostgreSQL	MariaDB
공식 지원	지원	지원	지원
Canonical 의미	공통	공통	공통
Vendor Render	oracle	postgresql	mariadb
검증	DDL·Runtime	DDL·Runtime	DDL·Runtime

Generated Domain 구조와 Generator-owned/User-owned 경계의 지원 조건을 확인합니다.

[Source](#) [길찾기](#) [cpf-tools/generator/contracts/cpf-domain.schema.json](#)

9. Performance·Compatibility·Release

9.1 Performance/Capacity

성능 판단은 TPS 하나가 아니라 latency, DB lock/query, 외부 연계 backlog, Worker 수, resource saturation 을 함께 봅니다. 용량 변경은 장애 격리와 replay 폭증 가능성까지 확인합니다.

- 대량 작업은 샘플 성공이 아니라 최대/경계 규모에서 측정합니다.
- Scale-out 시 shared state 와 idempotency/lease 가 다중 인스턴스를 견뎌야 합니다.

9.2 Compatibility

Compatibility 는 Java·Spring·Starter·Provider·DB·Generated output 조합이 같은 Public Contract 를 유지하는지 검증합니다.

- 지원 조합 변경은 consumer compile/runtime 과 generated output 까지 회귀 검증합니다.

9.3 Build/Test

Build/Test 는 root configuration 부터 consumer compile, unit/integration/runtime, publication 산출물까지 같은 Source identity 로 검증합니다.

- 실행 명령·ExitCode·테스트 결과와 기준 Source 를 같은 Evidence 에 연결합니다.

9.4 Release Acceptance

Release Acceptance 는 root build/test, Public API compatibility, DB3/Generator parity, publication 산출물과 Runtime 검증이 같은 Source identity 에서 통과했는지 확인합니다. 미실행 검증은 Release PASS 근거로 사용하지 않습니다.

- root build/test, Public API compatibility, DB3/Generator parity 와 publication 산출물이 같은 Source identity 를 가리켜야 합니다.
- Runtime 검증을 수행하지 못한 항목은 PASS 가 아니라 미검증으로 분리하고 재실행 조건을 남깁니다.

Release 후보가 충족해야 할 build/runtime/compatibility 검증 기준을 확인합니다.

표 9-1. 항목 / 지원/기준 / 제약 / 확인

비교·선택·확인에 필요한 항목만 압축해 정리합니다.

항목	지원/기준	제약	확인
Java	25	프로젝트 baseline 일치	gradle/cpf-stack.properties
Gradle	9.1.0	Wrapper 기준	gradle/cpf-stack.properties
Spring Boot	4.1.0	CPF baseline	gradle/cpf-stack.properties
DB	Oracle·PostgreSQL·MariaDB	DB3 only	cpf-tools/db/vendor

정상·실패·복구까지 통과해야 할 시나리오를 확인합니다.

Source [길찾기 cpf-tools/verification/tools/run-cpf-development-final-gate.py](https://github.com/cpf-tools/verification/tools/run-cpf-development-final-gate.py)